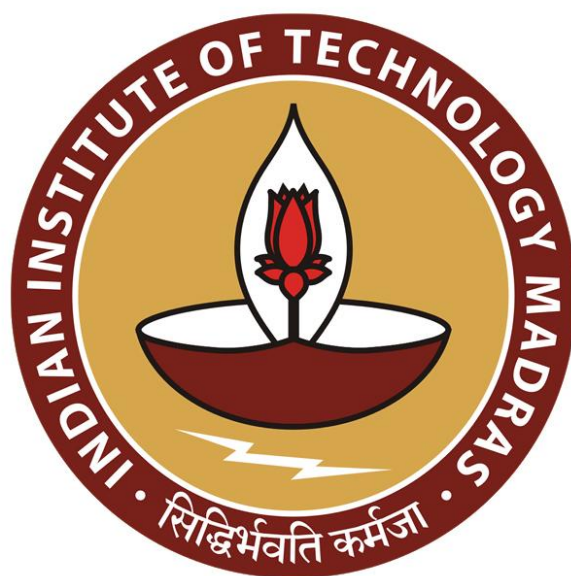
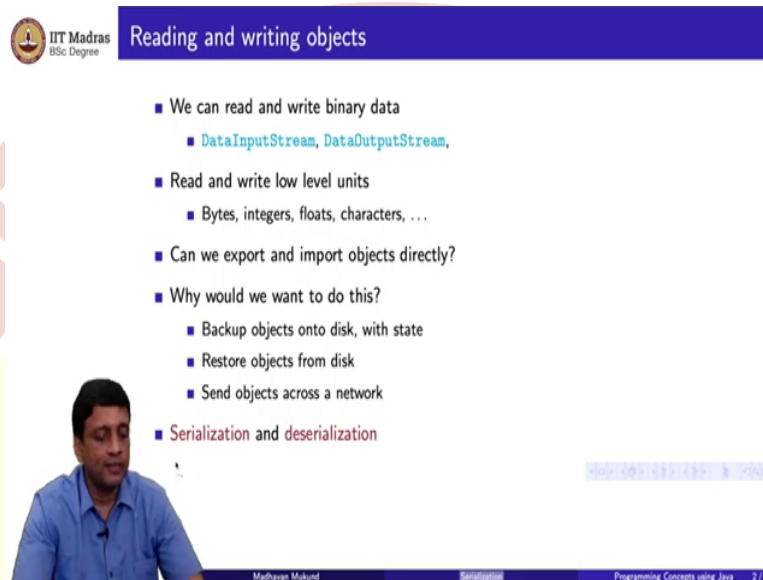




IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor. Madhavan Mukund
Department of Computer Science
Indian Institute of Technology, Madras
Week 9
Serialization

(Refer Slide Time: 00:14)



Reading and writing objects

- We can read and write binary data
 - `DataInputStream`, `DataOutputStream`,
- Read and write low level units
 - Bytes, integers, floats, characters, ...
- Can we export and import objects directly?
- Why would we want to do this?
 - Backup objects onto disk, with state
 - Restore objects from disk
 - Send objects across a network
- **Serialization and deserialization**

Madhavan Mukund Serialization Programming Concepts using Java 2 / 7

So we saw that we could read and write binary data using this `DataInputStream` and the `DataOutputStream`, but these streams read low level units, we can read bytes, integers, floats, characters and so on, but in java the data types also include those that the user defines and these are objects, they are large compilations which encapsulate various instance variables inside them.

So, it will be very tedious if we have to disassemble an object and write its component parts and rewrite it back, so the question is can we import and export objects directly. So, why would we want to do this? One reason is that we might want to actually save the state of a program on a disk; we might want to save the objects that are currently running so that we can restore the state and run later on.

So, if we want to do that then every object which is currently part of our running program has to be saved with the current values of its instance variables, also we might have a distributed application where things are running at different places, it could be an internet app or

something so we might want to take an object that is created at one location and pass it to another location.

So, there are various situations where it is convenient to be able to use objects as the unit of reading and writing and this is not directly available to us through this `DataInputStream` and `DataOutputStream`. So, this ability to read and write objects as a whole is called serialization, so writing an object is called serializing the object and reading the object that is taking an object which has been written and taking it and reading it back and reconstructing the object is called deserialization, and we will see a little later why these words come in.

(Refer Slide Time: 02:05)

The slide is titled "Reading and writing objects ...". It features the IIT Madras BSc Degree logo in the top left corner. The main content area contains a bullet point: "To write objects, Java has another output stream type, `ObjectOutputStream`". To the right of the text, there is a code snippet:

```
var out = new ObjectOutputStream(  
    new FileOutputStream("employee.dat"));
```

 Below the text and code, there is a small video feed of a man in a blue shirt. At the bottom of the slide, there is a footer with the text "Madhavan Mahand", "Serialization", and "Programming Concepts using Java 3 / 7".

The slide is titled "Reading and writing objects ...". It features the IIT Madras BSc Degree logo in the top left corner. The main content area contains two bullet points: "To write objects, Java has another output stream type, `ObjectOutputStream`" and "Use `writeObject()` to write out an object". To the right of the text, there is a code snippet:

```
var out = new ObjectOutputStream(  
    new FileOutputStream("employee.dat"));  
  
var emp = new Employee(...);  
var boss = new Manager(...);  
out.writeObject(emp);  
out.writeObject(boss);
```

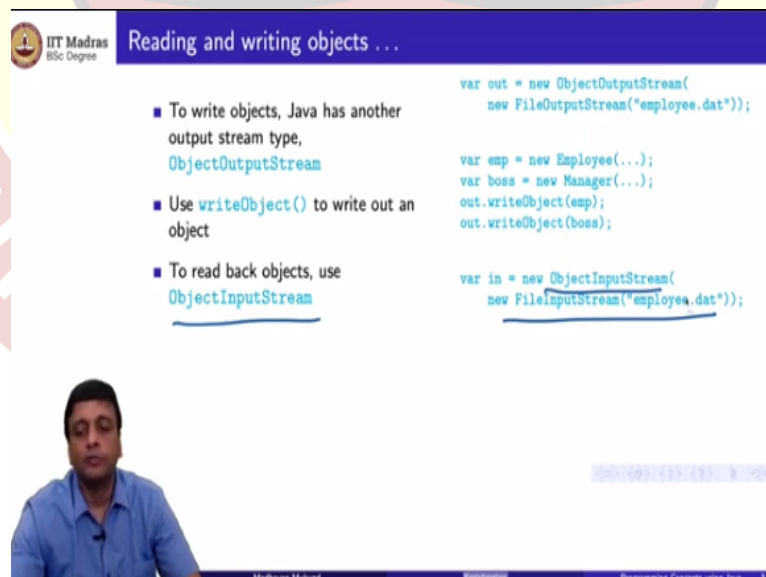
 Below the text and code, there is a small video feed of a man in a blue shirt. At the bottom of the slide, there is a footer with the text "Madhavan Mahand", "Serialization", and "Programming Concepts using Java 3 / 7".

As you would expect this is achieved by java providing a different type of input output stream. To write objects we use an `ObjectOutputStream`, so as before we will take for instance we want to write objects into a file we will first create a file output stream for that file name and we would wrap an object output stream, so very much the same way that we would take a print writer and wrap it around for a text output or a data output stream for a binary output we take an object output stream.

And now we have in this object output stream class a very convenient function called `writeObject`. So, if we have defined objects in our particular thing then we can just say write object and it will write out the object. So, the serialization we have to do, I mean we have to write it out but we can write out the entire so this constructor implies that there are various components in an employee object and a manager object, so without going through them one by one and writing them out explicitly this will write out all the data in an employee and in this manager object on to the disk.

And importantly, it will write some metadata about the objects. It is not just the raw data because then if I want to read it I would not know what the data is so it actually writes out some metadata about what these fields are.

(Refer Slide Time: 03:30)



IIT Madras
BSc Degree

Reading and writing objects ...

- To write objects, Java has another output stream type, `ObjectOutputStream`
- Use `writeObject()` to write out an object
- To read back objects, use `ObjectInputStream`

```
var out = new ObjectOutputStream(  
    new FileOutputStream("employee.dat"));  
  
var emp = new Employee(...);  
var boss = new Manager(...);  
out.writeObject(emp);  
out.writeObject(boss);  
  
var in = new ObjectInputStream(  
    new FileInputStream("employee.dat"));
```

Madhavan Mahalingam Serialisation Programming Concepts using Java 3 / 7



Reading and writing objects ...

- To write objects, Java has another output stream type, `ObjectOutputStream`
- Use `writeObject()` to write out an object
- To read back objects, use `ObjectInputStream`
- Retrieve objects in the same order they were written, using `readObject()`

```

var out = new ObjectOutputStream(
    new FileOutputStream("employee.dat"));

var emp = new Employee(...);
var boss = new Manager(...);
out.writeObject(emp);
out.writeObject(boss);

var in = new ObjectInputStream(
    new FileInputStream("employee.dat"));

var e1 = (Employee) in.readObject();
var e2 = (Employee) in.readObject();

```



Madhavan Mukund

Serialization

Programming Concepts using Java 3/7



Reading and writing objects ...

- To write objects, Java has another output stream type, `ObjectOutputStream`
- Use `writeObject()` to write out an object
- To read back objects, use `ObjectInputStream`
- Retrieve objects in the same order they were written, using `readObject()`
- Class has to allow serialization — implement marker interface `Serializable`

```

var out = new ObjectOutputStream(
    new FileOutputStream("employee.dat"));

var emp = new Employee(...);
var boss = new Manager(...);
out.writeObject(emp);
out.writeObject(boss);

var in = new ObjectInputStream(
    new FileInputStream("employee.dat"));

var e1 = (Employee) in.readObject();
var e2 = (Employee) in.readObject();

public class Employee
    implements Serializable {...}

```



Madhavan Mukund

Serialization

Programming Concepts using Java 3/7

So, to symmetrically get it back I have an object input stream, so again I wrap an object input stream say around a file input stream so that I can read back objects which had been written into that file earlier by this object output stream. To retrieve these objects I use read object, so read object will retreat and it is a sequential process so if I wrote this and wrote this then what the first e1 will correspond to the object emp and e2 will correspond to the object boss.

So, in order to use serialization the class has to allow it, so there is what is called a marker interface, so remember we talked about clonable so if a if an object can be cloned the the class of the object has to implement clonable. It does not have any requirements in terms of functions it is just a requirement in terms of indicating that this possibility exists called a

marker interface. Here, we have a marker interface called serializable which indicates that this class permits its objects to be serialized or written out using this object output stream.

(Refer Slide Time: 04:33)

IIT Madras BSc Degree

How serialization works

- `ObjectOutputStream` examines all the fields and saves their contents
- `ObjectInputStream` "reconstructs" the object, effectively calls a constructor
- What happens when many objects share the same object as an instance variable?

```
class Manager extends Employee {  
    private Employee secretary;  
    ....  
}
```
- Two managers have the same secretary
- How do we avoid duplicating objects when serializing?

Madhavan Mukund Serialization Programming Concepts using Java 4 / 7

IIT Madras BSc Degree

How serialization works

- `ObjectOutputStream` examines all the fields and saves their contents
- `ObjectInputStream` "reconstructs" the object, effectively calls a constructor
- What happens when many objects share the same object as an instance variable?

```
class Manager extends Employee {  
    private Employee secretary;  
    ....  
}
```
- Two managers have the same secretary
- How do we avoid duplicating objects when serializing?
- Each object is assigned a serial number — hence, **serialization**
 - When first encountered, save the data to output stream
 - If saved previously, record serial number
 - Reverse the process when reading

Madhavan Mukund Serialization Programming Concepts using Java 4 / 7

So, how does serialization work? So, an `ObjectOutputStream` examines all the fields and writes out their content, so it actually writes out a description of the object, it is not just a binary dump of the object it writes out the description of the object. So, when you construct an object back it effectively is like creating an object in the normal sense. You normally create an object calling a constructor and passing the argument, so more or less the same thing happens here in object inputs stream reconstructs the object by effectively calling a constructor.

So, there is an obvious problem here, so what if we have within one object a nested object? For example, supposing the manager class associates with every manager object another employee object who is the secretary assigned to this manager. Now in an organization it is very possible that two managers have the same secretary working for them. So, if we now try to serialize a manager object it will store a copy of the secretary object, because that is a nested object inside the manager, but now the second manager who has the same secretary will store a duplicate copy of the secretary object.

So, we will have the secretary object also will exist somewhere then we will have the original secretary object because there is an employee who is a secretary so is presumably represented by an employee object somewhere else and that employee object is nested inside both the manager and the other manager.

So, to avoid this duplication, what happens inside the serializable interface which is something that we do not have to worry about but this write object actually implicitly assigns a serial number to every object that it encounters in the process of writing. So, the first time when it writes out an object of course it associates a serial number and it actually writes it out.

The second time when it sees an object if it has already been written out it does not write out the object in, it just mentions that here is a placeholder and this placeholder corresponds to this serial number. So, this serial number is the object which I have already written go and look it up, and when you are reading you reverse the process.

So the first time, remember the first time you will read the objects in the same order in which they are written, the first time when you recover an object with a serial number you will actually get a concrete object reference in your code and then henceforth when you see a reference to an object which you have already read you will replace that serial number by pointing to this object that you have already read.

So this use of serial numbers to avoid duplication is actually the origin of the term serialization so this is why this process of storing and restoring objects as objects is called serialization.

(Refer Slide Time: 07:17)



IIT Madras
BSc Degree

Customizing serialization

- Some objects should not be serialized
— values of file handles, ...
- Mark such fields as transient

```
public class LabeledPoint
    implements Serializable{
    private String label;
    private transient Point2D.Double point;
    ...
}
```

Madhavan Mukund Serialization Programming Concepts using Java 5/7



IIT Madras
BSc Degree

Customizing serialization

- Some objects should not be serialized
— values of file handles, ...
- Mark such fields as transient
- Can override writeObject()
 - defaultWriteObject() writes out the object with all non-transient fields
 - Then explicitly write relevant details of transient fields

```
private void
    writeObject(ObjectOutputStream out)
        throws IOException{
    out.defaultWriteObject();
    out.writeDouble(point.getX());
    out.writeDouble(point.getY());
}
```

Madhavan Mukund Serialization Programming Concepts using Java 5/7



- Some objects should not be serialized
— values of file handles, ...
- Mark such fields as `transient`
- Can override `writeObject()`
 - `defaultWriteObject()` writes out the object with all non-transient fields
 - Then explicitly write relevant details of transient fields
- ...and `readObject()`
 - `defaultReadObject()` reconstructs object with all non-transient fields
 - Then explicitly reconstruct transient fields

```
private void
writeObject(ObjectOutputStream out)
throws IOException{
    out.defaultWriteObject();
    out.writeDouble(point.getX());
    out.writeDouble(point.getY());
}

private void
readObject(ObjectInputStream in)
throws IOException {
    in.defaultReadObject();
    double x = in.readDouble();
    double y = in.readDouble();
    point = new Point2D.Double(x, y);
}
```



Some objects should not be serialized, so supposing as a part of the runtime of your thing you have associated something connected to a file. Now when you reconstruct the object that file may or may not exist, so this connection to the file like a file handle is not something that should be restored. So, how do you control this? So, you can mark some fields as transient, if you mark something as transient this is a new modifier it says that this field should be skipped when serializing.

For example here it has we have a class called labeled point it allows serialization but it has a field inside it which is a point from some other class, which it marks as transient, so when we serialize the label point this field will not get copied out. If we want to copy out faithfully this thing we have to then do something ourselves, so we have to rewrite or override the write object. So, this is the signature of write object in general which is given in that outputs class the object output stream that we saw before, so we override it.

What we do is we first call this default write object. So, what is default write object do? It basically does what write object would have been done in the original class, so it writes out this object with all its non transient fields as though it was serializing it. Now we are left with the transient fields and that we have to do manually, so we take the point that is not written out because it was marked transient and we store the relevant data for that point manually using this write double.

This is a basically a binary write like it was available in the data output stream. Now when we read the thing we have to do the same thing, first we reconstruct so we have to first of all

redefine this read object, so we reconstruct the object using the default read object which will reconstruct the original object, but in that original object there will be this point field which has no value because it was transient and therefore not saved.

So now, we read the remaining two doubles that we read and we explicitly call a constructor for that class to associate a correct reference for this field point. When we have these transient fields then we use our own customize write object and read object to achieve serializability but it is under our control.

(Refer Slide Time: 09:48)

IIT Madras
BSc Degree

Handle with care!

- Serialization is a good option to share data within an application
- Over time, older serialized objects may be incompatible with newer versions
 - Some mechanisms for version control, but still some pitfalls possible
- Deserialization implicitly invokes a constructor
 - Running code from an external source
 - Always a security risk

Madhavan Mukund Serialization Programming Concepts using Java 6 / 7

So, serialization clearly something which java feels is tricky because it forces you to announce that you implement the serializable interface. It is generally okay if you are serializing for your own purposes like a backup and a restore because you know what you are doing, but if you are sharing data with other developers or if you are sharing data across a network or something, then there are several problems.

One is of course that this breaks encapsulation to the extent that if you have serialized an old version of your object and then you change your code and somebody now works with the older versions of the objects after deserializing there is a mismatch between the objects they are using and the objects that you are currently manufacturing.

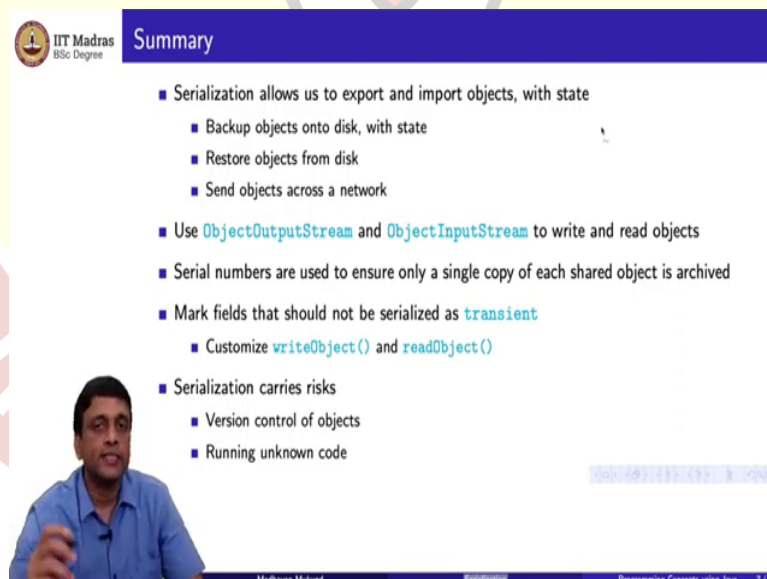
So, there is a problem of versioning you can actually build in some version control into serialization but its not very straight forward and even there there are some pitfalls, so you

have to be a little careful when you try to understand the implications of where all your serialized objects are going to be used.

Another issue with serialization is specific to deserialization. What is the serialization do? It means that you take an object which is given to you and you reconstruct it, so implicitly you are calling some kind of constructor function on this object. Now as anybody who has been exposed to the internet knows it is always dangerous to take something that somebody else gives you some piece of code that somebody unknown gives you and run it. It may be something which is going to damage your system, so it is always a security risk to run some unknown code on your system.

Of course if you are doing it in the first situation here where you are just recovering a backup that you took you know exactly what is happening, but if you are taking an object which is constructed by some other developer and you are deserializing it you should be aware of the risk that you are taking in running code with somebody else's package and which you have no access to before you run it because it comes automatically as a part of that read object thing.

(Refer Slide Time: 11:50)



IIT Madras
BSc Degree

Summary

- Serialization allows us to export and import objects, with state
 - Backup objects onto disk, with state
 - Restore objects from disk
 - Send objects across a network
- Use `ObjectOutputStream` and `ObjectInputStream` to write and read objects
- Serial numbers are used to ensure only a single copy of each shared object is archived
- Mark fields that should not be serialized as `transient`
 - Customize `writeObject()` and `readObject()`
- Serialization carries risks
 - Version control of objects
 - Running unknown code

Madhavan Mukund Serialization Programming Concepts using Java 7/7

So, to summarize, with serialization we can export and import objects directly with their state, so it is not just binary data at a low level of ints and floats and doubles, but the entire object can be sent out and restored and this is useful for backing up and restoring. It is also useful for sharing data across a network if we have a distributed application. So, we use this

object output stream and object input stream as wrappers around our file stream in order to write and read objects.

So internally, this problem of duplicating shared objects is resolved by using serial numbers and the serial number is the reason why this is called serialization. We can choose to not serialize certain fields by marking them as transient and then we can save some information about these transient fields manually by customizing the write object and read object methods for that class.

So, remember that serialization should be done with care because it carries risks. If the objects that you are working with change in your code the serialized objects will remain in the old version and they will be incompatible and when you deserialize an object written by somebody else you carry the risk always associated with running code which is not known to you, so externally developed code if you run on your system then you expose yourself to whatever risk that might entail you.

